

Object Oriented Programming Introduction

Lecture 1 **Spring 2025**

Lecture Contents

- C++ Programming basics
- Control structures
- Arrays

Programming Style

- C++ is a free-format language, which means that:
 - Extra blanks (spaces) or tabs before or after identifiers/operators are ignored
 - Blank lines are ignored by the compiler just like comments
 - Code can be indented in any way
 - There can be more than one statement on a single line
 - A single statement can continue over several lines

Programming Style (cont.)

- In order to improve the readability of your program, use the following conventions:
 - Start the program with a **header** that tells what the program does
 - Use **meaningful variable names and Camel notation**
 - **Document** each variable declaration with a comment telling what the variable is used for
 - Place each **executable statement** on a **single line**
- A segment of code is a sequence of executable statements that belong together
 - Use blank lines to separate different segments of code
 - Document each segment of code with a comment telling what the segment does.

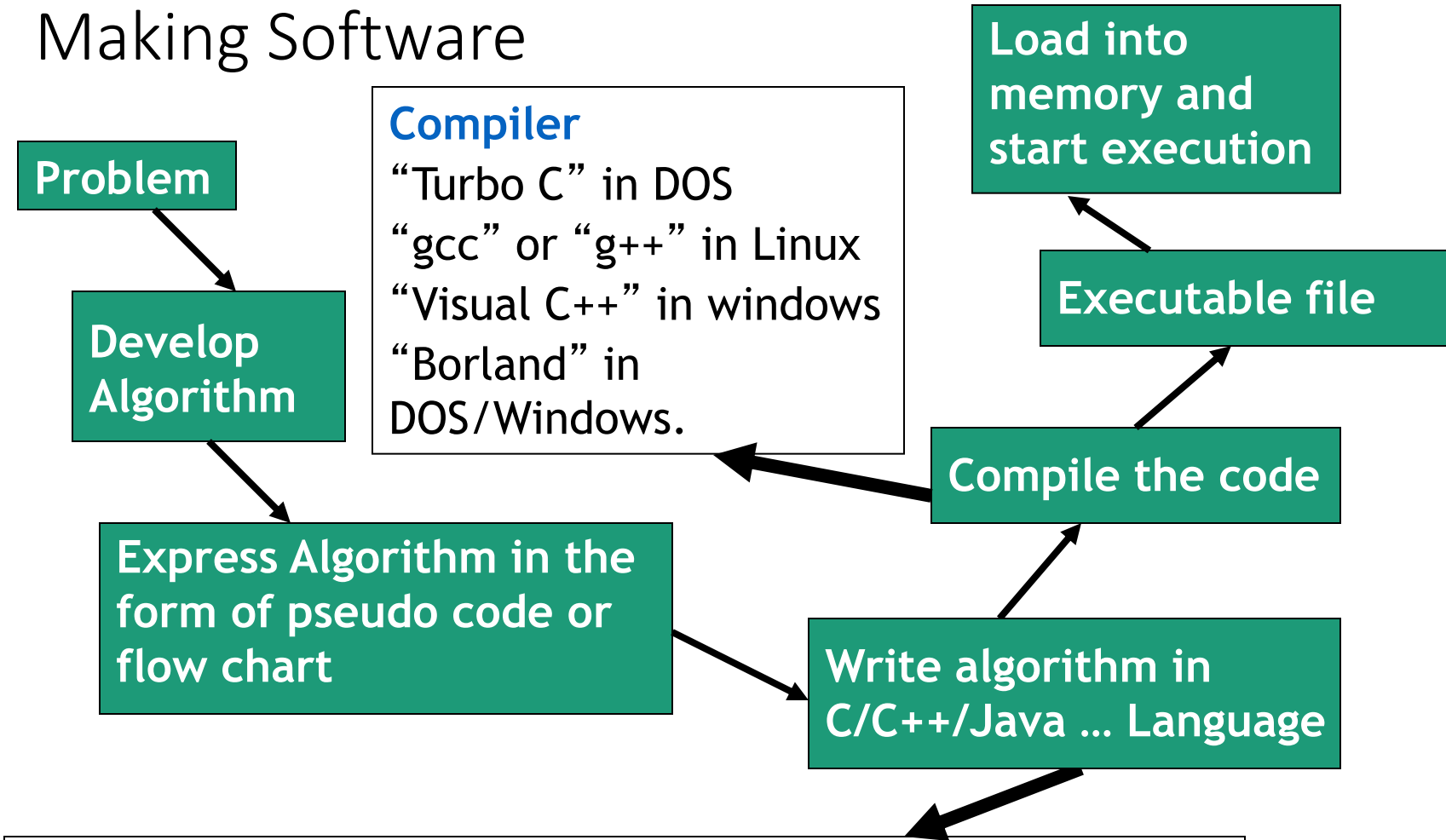
Structure of a C++ Program

- A C++ program is a collection of definitions and declarations:
 - data type definitions
 - global data declarations
 - function definitions (subroutines)
 - class definitions
 - a special function called
 - **main()** (where the action starts)

C++ program execution process

- How many stages does a c++ program goes through before it is executed?

Making Software



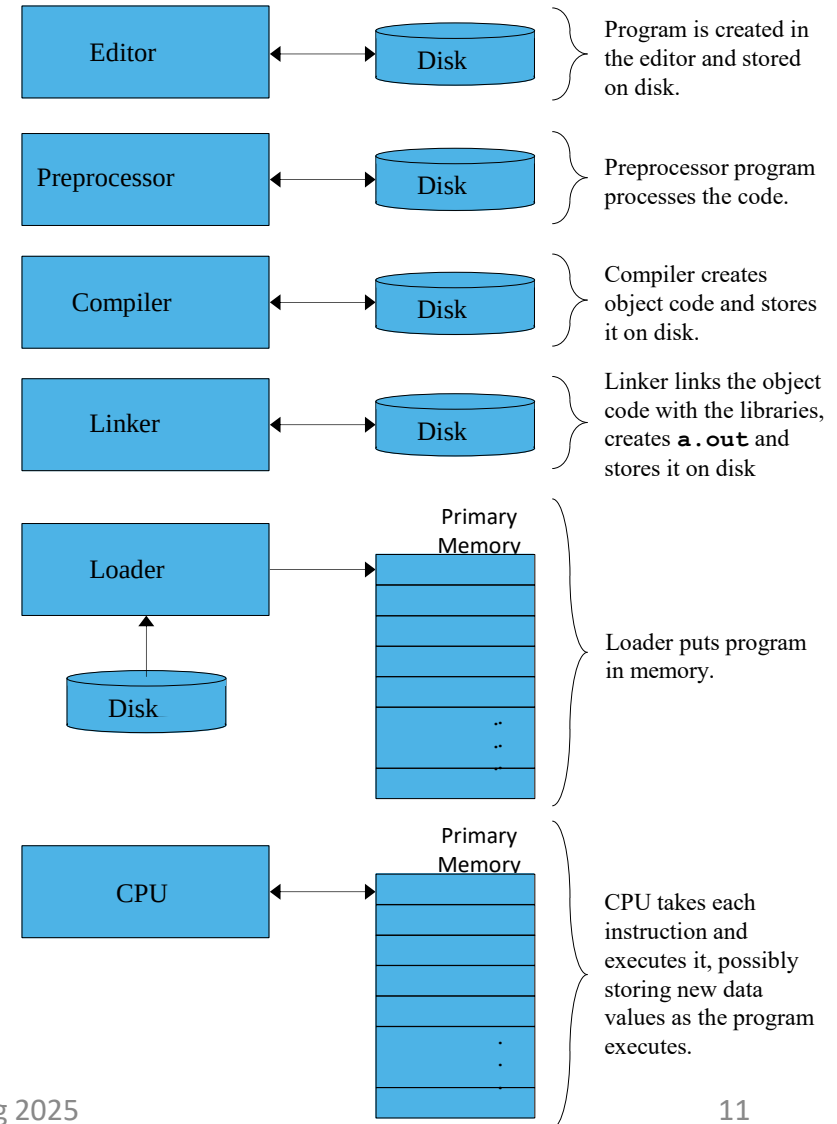
Writing code in “C++” language means, we must have to follow a certain rules of “C++” language.

This is called syntax of “C++” language.

Basics of a Typical C++ Environment

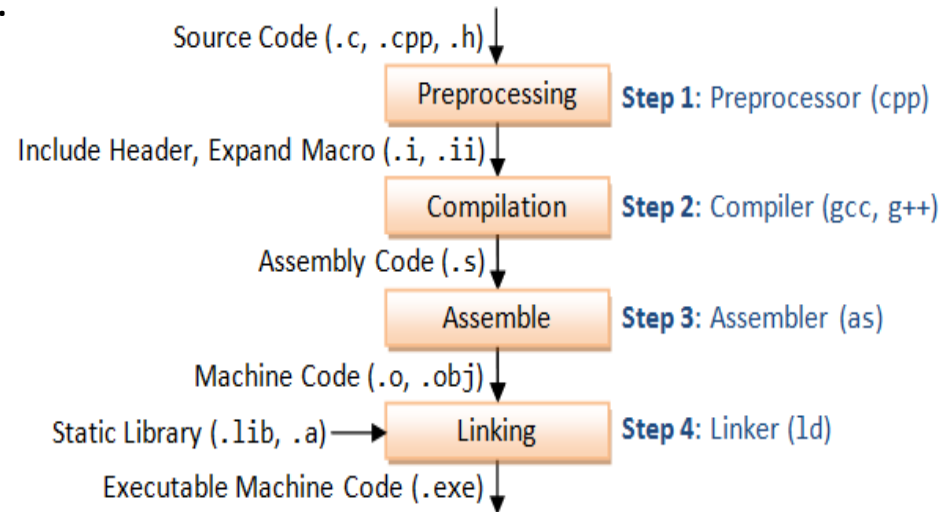
Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



C++ Compilation Process

- Compiler compiles a C/C++ program into executable in 4 steps as shown in the diagram. For example, a "gcc -o hello.exe hello.c" is carried out as follows:
- **Pre-processing:** via the GNU C Preprocessor (cpp.exe), which includes the headers (#include) and expands the macros (#define).
- The resultant intermediate file "hello.i" contains the expanded source code.
- **Compilation:** The compiler compiles the pre-processed source code into assembly code for a specific processor.
 - The resultant assembly file is "hello.s".
- **Assembly:** The assembler (as.exe) converts the assembly code into machine code in the object file "hello.o".
- **Linker:** Finally, the linker (ld.exe) links the object code with the library code to produce an executable file "hello.exe".



C++ Programming Examples

- Following are several examples
 - The examples illustrate many important features of C++
 - Each example is analyzed one statement at a time.

```
1 // Fig. 1.2: fig01_02.cpp
2 // A first program in C++
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome to C++!\n";
8
9     return 0;    // indicate that program terminated normally
10 }
```

Comments

Written between `/*` and `*/` or following a `//`.

Improve program readability and do not cause the computer to perform any action.

preprocessor directive

Message to the C++ preprocessor.

Lines beginning with `#` are preprocessor directives.

`#include <iostream>` tells the preprocessor to include the `iostream` header file.

C++ programs contain one or more functions, one of which must be **main**

Parenthesis are used to indicate a function

integer value.

return is a way to exit a function from a function.

return 0, in this case, means that the program terminated normally.

Prints the *string* of characters contained between the

including **std::cout**, the `<<`

ing **"Welcome to C++!\n"** and

), is called a *statement*.

All statements must end with a semicolon.

every function

A Simple Program: Printing a Line of Text

- **std::cout**
 - Standard output stream object
 - “Connected” to the screen
 - **std::** specifies the "namespace" which **cout** belongs to
 - **std::** can be removed through the use of **using** statements
- **<<**
 - Stream insertion operator
 - Value to the right of the operator (right operand) inserted into output stream (which is connected to the screen)
 - **std::cout << “Welcome to C++!\n”;**
- ****
 - Escape character
 - Indicates that a “special” character is to be output

A Simple Program: Printing a Line of Text

Escape Sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\"</code>	Double quote. Used to print a double quote character.

- There are multiple ways to print text
 - Following are more examples

```
1 // Fig. 1.4: fig01_04.cpp
2 // Printing a line with multiple statements
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9
10    return 0;    // indicate that program ended successfully
11 }
```

Welcome to C++!

Unless new line '**\n**' is specified, the text continues on the same line.

Another Simple Program: Adding Two Integers

- Variables
 - Location in memory where a value can be stored for use by a program
 - Must be declared with a name and a data type before they can be used
 - Some common data types are:
 - `int` - integer numbers
 - `char` – characters
 - Double – floating point numbers
 - Example: `int myvariable;`
 - Declares a variable named `myvariable` of type `int`
 - Example: `int variable1, variable2;`
 - Declares two variables, each of type `int`

Another Simple Program: Adding Two Integers

- **>>** (stream extraction operator)
 - When used with `std::cin`, waits for the user to input a value and stores the value in the variable to the right of the operator
 - The user types a value, then presses the *Enter* (Return) key to send the data to the computer

- Example:

```
int myVariable;  
std::cin >> myVariable;
```

- Waits for user input, then stores input in `myVariable`

- **=** (assignment operator)
 - Assigns value to a variable
 - Binary operator (has two operands)
 - Example:

```
sum = variable1 + variable2;
```

```

1 // Fig. 1.6: fig01_06.cpp
2 // Addition program
3 #include <iostream>
4
5 int main()
6 {
7     int integer1, integer2, sum;           // declaration
8
9     std::cout << "Enter first integer\n"; //
10    std::cin >> integer1; ←
11    std::cout << "Enter second integer\n"; //
12    std::cin >> integer2;                 // read an integer
13    sum = integer1 + integer2;             // assignment of sum
14    std::cout << "Sum is " << sum << std::endl; // print sum
15
16    return 0; // indicate that program ended success
17 }

```

Notice how **std::cin** is used to get user input.

std::endl flushes the buffer and prints a newline.

Variables can be output using **std::cout << variableName**.

```

Enter first integer
45
Enter second integer
72
Sum is 117

```

Using declarations

- **using** statements
 - Eliminate the need to use the **std::** prefix
 - Allow us to write `cout` instead of **std::cout**
 - To use the following functions without the **std::** prefix, write the following at the top of the program

```
using std::cout;
```

```
using std::cin;
```

```
using std::endl;
```

```

1 // Fig. 1.14: fig01 14.cpp
2 // Using if statements, relational
3 // operators, and equality operators
4 #include <iostream>
5
6 using std::cout; // program uses cout
7 using std::cin;  // program uses cin
8 using std::endl; // program uses endl
9
10 int main()
11 {
12     int num1, num2;
13
14     cout << "Enter two integers, and I will tell you\n"
15           << "the relationships they satisfy: ";
16     cin >> num1 >> num2; // read
17
18     if ( num1 == num2 )
19         cout << num1 << " is equal to " << num2 << endl;
20
21     if ( num1 != num2 )
22         cout << num1 << " is not equal to " << num2 << endl;
23
24     if ( num1 < num2 )
25         cout << num1 << " is less than " << num2 << endl;
26
27     if ( num1 > num2 )
28         cout << num1 << " is greater than " << num2 << endl;
29
30     if ( num1 <= num2 )
31         cout << num1 << " is less than or equal to "
32           << num2 << endl;
33

```

Notice the **using** statements.

Enter two integers, and I will tell you
the relationships they satisfy: 3 7

The **if** statements test the truth of the condition. If it is **true**, the body is executed. If not, body is skipped.

To include multiple statements in a body, delineate them with braces **{}**.

3 is less than or equal to 7

```

34     if ( num1 >= num2 )
35         cout << num1 << " is greater than or equal to "
36             << num2 << endl;
37
38     return 0;    // indicate that program ended successfully
39 }

```

```

Enter two integers, and I will tell you
the relationships they satisfy: 3 7
3 is not equal to 7
3 is less than 7
3 is less than or equal to 7

```

```

Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12

```

```

Enter two integers, and I will tell you
the relationships they satisfy: 7 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7

```

Class Activity#1

- Write a program that generates the following output, using only one *cout statement* and *proper spacing*

1900	2135
1950	2235
2000	2335
2050	2435

Control Structures

- How many control structures are there?

Control Structures

- Normally, statements in a program execute one after the other in the order in which they're written.
 - Called **sequential execution**.
- Various C++ statements enable you to specify that the next statement to execute may be other than the next one in sequence.
 - Called **transfer of control**.
- All programs could be written in terms of only three **control structures**
 - the **sequence structure**
 - the **selection structure** and
 - the **repetition structure**

Control Structures (if else and Switch)

- C++ provides three types of **selection statements**
 - The **if selection** statement either performs (selects) an action if a condition (predicate) is true or skips the action if the condition is false.
 - The **if...else selection** statement performs an action if a condition is true or performs a different action if the condition is false. Use if-else when the conditions involve *complex logical expressions* or *comparisons*.
 - The **switch selection** statement performs one of many different actions, depending on the value of an integer expression. Use switch-case when checking against discrete, constant values (e.g., integer, character).

Control Structures (if else and Switch)

- The if selection statement is a **single-selection statement** because it selects or ignores a *single action* (or, as we'll soon see, a *single group of actions*).
- The if...else statement is called a **double-selection statement** because it selects between two different actions (or groups of actions).
- The switch selection statement is called a **multiple-selection statement** because it selects among many different actions (or groups of actions).

Control Structures (if selection statement)

- Programs use selection statements to choose among alternative courses of action.
- For example, whether “student’s grade is greater than or equal to 60” is `true` or `false`.

```
if ( grade >= 60 )  
    cout << "Passed";
```

- If `true`, “Passed” is printed and the next pseudocode statement in order is “performed”
- If `false`, the print statement is ignored and the next statement in order is performed.

Control Structures (if-else selection statement)

- Nested `if...else` statements test for multiple cases by placing `if...else` selection statements inside other `if...else` selection statements.

```
if ( studentGrade >= 90 ) // 90 and above gets "A"
    cout << "A";
else
    if ( studentGrade >= 80 ) // 80-89 gets "B"
        cout << "B";
    else
        if ( studentGrade >= 70 ) // 70-79 gets "C"
            cout << "C";
        else
            if ( studentGrade >= 60 ) // 60-69 gets "D"
                cout << "D";
            else // less than 60 gets "F"
                cout << "F";
```

Control Structures (Switch Selection Statement)

- Another alternative to if—else statement is Switch statement.
- The idea behind a switch statement is simple:
 - an expression (sometimes called the condition) is evaluated to produce a value.
 - If the expression's value is equal to the value after any of the case labels, the statements after the matching case label are executed.
 - If **NO** matching value can be found and a **default label** exists, the statements after the default label are executed instead.

Control Structures (Switch Selection Statement)

```
int main(){
    int num;
    cout<<"Please Enter a number: ";
    cin>>num;
    switch (num)    {
        case 1:
            cout << 1 << '\n';
            break;
        case 2:
            cout << 2 << '\n';
            break;
        case 3:
            cout << 3 << '\n';
            break;
        default:
            cout<<"Default Case: "<<'\n';
    }
    std::cout<<"Switch Terminated: "<<std::endl;
    return 0;}
```

Class activity

- Will the below code be executed?
- What is the output of the below code?

```
#include <iostream>
using namespace std;
int main() {
    int x = //enter 1-3;
    switch (x) {
        case 1:
        case 2:
        case 3:
            cout << "Matched case 1, 2, or 3" << endl;
            break;
        default:
            cout << "Default case" << endl;
    }
    return 0;
}
```

Control Structures (repetition statements)

- C++ provides three types of repetition statements (also called **looping statements** or **loops**) for performing statements repeatedly while a condition (called the **loop-continuation condition**) remains true.
 - These are the **while**, **do...while** and **for** statements.
- The **while** and **for** statements perform the action (or group of actions) in their bodies zero or more times.
- The **do...while** statement performs the action (or group of actions) in its body *at least once*.

Control Structures (While repetition Statement)

- A **repetition statement** (also called a **looping statement** or a **loop**) allows you to specify that a program should repeat an action while some condition remains true.
 - The statement contained in the *While* repetition statement constitutes the body of the *While*, which can be a single statement or a block.
 - Eventually, the condition will become false, the repetition will terminate, and the first pseudocode statement after the repetition statement will execute.

Control Structures (while repetition Statement)

- Consider a program segment designed to find the first power of 3 larger than 100. Suppose the integer variable product has been initialized to 3.
- When the following while repetition statement finishes executing, product contains the result:

```
int product = 3;  
  
while ( product <= 100 )  
    product = 3 * product;
```

Control Structures (Do while Statement)

- A do while statement is a looping construct that works just like a while loop, except the statement always executes at least once.
- After the statement has been executed, the do-while loop checks the condition. If the condition evaluates to true, the path of execution jumps back to the top of the do-while loop and executes it again.

do

```
statement; // can be a single statement or a compound  
statement
```

```
while (condition);
```

Control Structures (Do while Statement)

```
int main() {  
    // selection must be declared outside of the do-while so we can use it  
    later  
    int selection;  
    do {  
        std::cout << "Please make a selection: \n";  
        std::cout << "1) Addition\n";  
        std::cout << "2) Subtraction\n";  
        std::cout << "3) Multiplication\n";  
        std::cout << "4) Division\n";  
        std::cin >> selection;    }  
    while (selection != 1 && selection != 2 &&  
        selection != 3 && selection != 4);  
    // do something with selection here  
    // such as a switch statement  
    std::cout << "You selected option #" << selection << '\n';  
    return 0; }
```

class activity: Control Structures (For Statement)

- Write a program that gives the following output:

```
i = 0, j = 0  
i = 0, j = 1  
i = 0, j = 2  
i = 0, j = 3  
i = 0, j = 4  
i = 1, j = 0  
i = 1, j = 1  
i = 1, j = 2  
i = 1, j = 3  
i = 1, j = 4  
i = 2, j = 0  
i = 2, j = 1  
i = 2, j = 2  
i = 2, j = 3  
i = 2, j = 4  
i = 3, j = 0  
i = 3, j = 1  
i = 3, j = 2  
i = 3, j = 3  
i = 3, j = 4
```

Control Structures (For Statement)

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 0; i < 5; ++i) {
        for (int j = 0; j < 5; ++j) {
            cout << "i = " << i << ", j = " << j << endl;
        }
    }
    return 0;
}
```

Control Structures (For Statement)

- Now modify the code on the previous slide such that it gives the below output:

i = 0, j = 0

i = 0, j = 1

i = 0, j = 2

i = 0, j = 3

i = 0, j = 4

i = 1, j = 0

i = 1, j = 1

i = 1, j = 2

i = 1, j = 3

i = 1, j = 4

i = 2, j = 0

i = 2, j = 1

i = 2, j = 2

i = 2, j = 3

Exited nested loops.

Control Structures (For Statement)

```
#include <iostream>
using namespace std;
int main() {
    bool exitLoops = false; // Flag to break out of outer loop
    for (int i = 0; i < 5; ++i) {
        for (int j = 0; j < 5; ++j) {
            if (i == 2 && j == 3) {
                exitLoops = true;
                break; // Break out of the inner loop
            }
            cout << "i = " << i << ", j = " << j << endl;
        }
        if (exitLoops) break; // Check the flag and break the outer loop
    }
    cout << "Exited nested loops." << endl;
    return 0;
}
```

Control Structures (For Statement)

- Write code for the below desired output:

Processing i = 0...

Processing i = 1...

Processing i = 2...

Processing i = 3...

Processing i = 4...

Control Structures (For Statement)

- Now modify your written code such that it gives the following output:

Processing i = 0...

Processing i = 1...

Skipped i = 2

Processing i = 3...

Processing i = 4...

Control Structures (For Statement)

- Now modify your written code such that it gives the following output:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i < 5; ++i) {
        if (i == 2) {
            cout << "Skipped i = " << i << endl;
            continue; // Skip the rest of the loop for `i = 2`
        }
        cout << "Processing i = " << i << endl;
    }
    return 0;
}
```

Output:

```
Processing i = 0...
Processing i = 1...
Skipped i = 2
Processing i = 3...
Processing i = 4...
```